

Technical Report: Min-Cost Flow Algorithms and Visualization

Thomas Lang 22500855

May 3, 2026

Contents

1	Project Overview and Objectives	2
1.1	Introduction	2
1.2	Core Requirements	2
1.3	Personal Objectives and Visual Features	3
1.4	Software Operation and Graph Import	3
2	Graph Architecture and Data Structures	4
2.1	The Adjacency List Structure	4
2.2	The Edge Data Model and Residual Logic	4
2.3	Visual Metadata Integration	4
2.4	The Memento Pattern for History Management	5
3	Replay Architecture and Execution Control	5
3.1	Snapshot-Based State Management	5
3.2	History Navigation and Memory Efficiency	5
3.3	Threading and the Execution Engine	6
3.4	The Controller	6
4	Algorithm Implementation and Logic	6
4.1	Maximum Flow and the Edmonds-Karp Variation	6
4.2	Minimum Cut Reachability Analysis	7
4.3	Cycle-Canceling for Cost Optimization	7
4.4	Successive Shortest Path and Node Potentials	7
5	Graphical User Interface (GUI)	9
5.1	Modular Layout and Panel Organization	9
5.2	The Graph Rendering Engine	9
5.3	Integrated Pedagogical Feedback	10

Chapter 1

Project Overview and Objectives

1.1 Introduction

The project focuses on the implementation and visualization of key network optimization problems, specifically Maximum Flow and Minimum Cost Flow. The software serves as an interactive educational tool designed to illustrate the step-by-step logic of flow algorithms through a graphical interface. By combining algorithmic rigor with visual feedback, the system allows for a deeper understanding of how residual networks and cost adjustments evolve during execution.

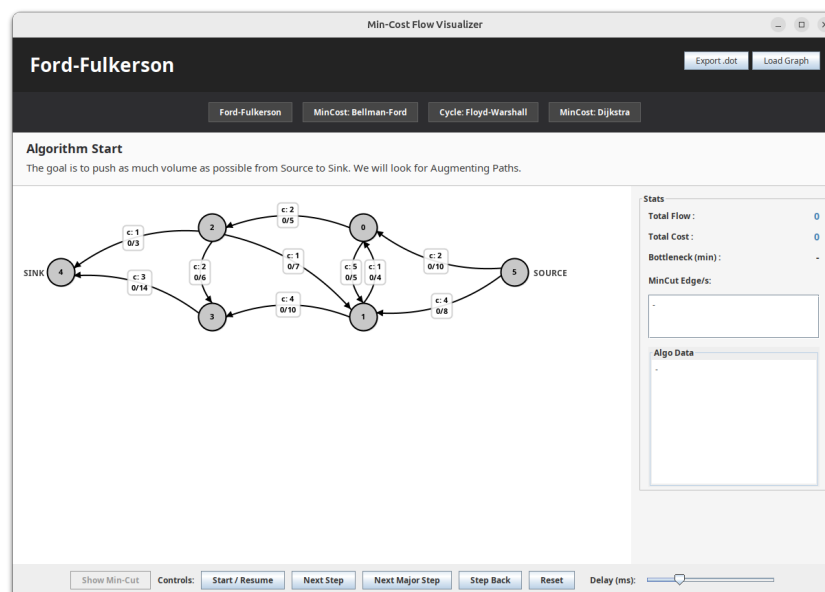


Figure 1.1: The visual interface displaying calculated flow and network statistics.

1.2 Core Requirements

The software fulfills several mandatory technical requirements. It implements the Ford-Fulkerson algorithm to calculate the maximum flow value, providing the specific flow on every edge and identifying the minimum cut. For cost optimization, the program utilizes successive shortest paths through two primary methods: negative cost cycle detection

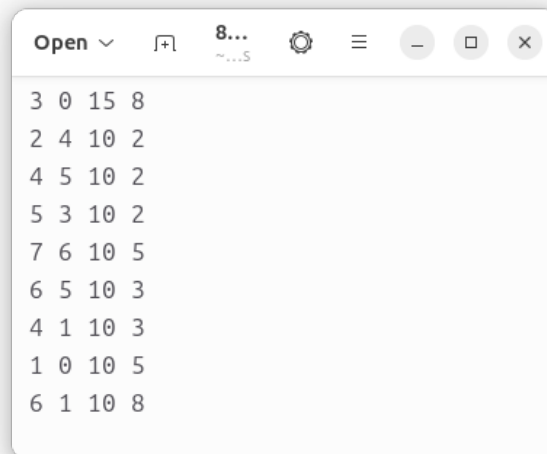
via Bellman-Ford and Floyd-Warshall, and efficient pathfinding via Dijkstra's algorithm using node potentials for cost renormalization. These implementations ensure that the final flow is optimal in terms of both volume and economy.

1.3 Personal Objectives and Visual Features

Beyond standard requirements, the project integrates a dynamic Graphical User Interface (GUI) to render graph structures and flow data. A robust execution control system enables users to pause, resume, and navigate backward or forward through the algorithm's history. Furthermore, an integrated explanation panel provides real-time textual descriptions of the logic being applied, such as identifying a bottleneck or visiting a specific node.

1.4 Software Operation and Graph Import

Running the program requires a standard Java environment. The entry point is located in `Main.java`, which initializes the controller. On startup, the system looks for a default graph file named `g1.txt` within a `Graphs` folder. Users can load custom networks using the "Load Graph" button in the header. To import a Graph, a `.txt` file must be chosen with a specific structure: the first line contains the number of nodes, number of arcs, source node, and sink node; the subsequent lines define the source node, destination node, capacity, and cost of each edge.



```
3 0 15 8
2 4 10 2
4 5 10 2
5 3 10 2
7 6 10 5
6 5 10 3
4 1 10 3
1 0 10 5
6 1 10 8
```

Figure 1.2: Example text format for graph definitions.

Chapter 2

Graph Architecture and Data Structures

2.1 The Adjacency List Structure

The core of the application relies on an adjacency list to represent the flow network. The `Graph` class maintains an array of lists, `List<Edge>[] adj`, where each index in the array corresponds to a node in the network. This structure is chosen because it allows for efficient traversal of outgoing edges during pathfinding operations, such as Breadth-First Search in Ford-Fulkerson or the priority queue operations in Dijkstra. Every node is identified by an integer index, and global network properties like the number of nodes, the source, and the sink are stored as primitive integers for quick access during algorithmic iterations.

2.2 The Edge Data Model and Residual Logic

The `Edge` class serves as the primary data container, storing both mathematical and visual information. Beyond standard fields like `capacity`, `flow`, and `cost`, the structure includes a `reverse` index. When the `addEdge` method is called, two edge objects are created: a forward edge with the specified capacity and cost, and a residual back-edge with zero initial capacity and a negative cost value. The `reverse` field stores the position of the back-edge in the destination node's adjacency list, which allows the algorithms to update residual capacities in $O(1)$ time during flow augmentation.

2.3 Visual Metadata Integration

To bridge the gap between the mathematical model and the graphical interface, the `Edge` class contains several boolean flags used for real-time rendering. The `isExploring` flag is toggled when an algorithm inspects an edge, while `isPath` is used to highlight the final path selected for flow augmentation. Additionally, the `isMinCut` flag identifies edges that belong to the bottleneck of the network after a maximum flow is reached. This design allows the `GraphPainter` to render state-specific colors and styles without requiring a separate mapping of the algorithm's progress.

2.4 The Memento Pattern for History Management

The framework implements a variation of the Memento pattern through the `GraphState` inner class to facilitate the replay functionality. A `GraphState` object acts as a copy snapshot of the network's current status. Instead of duplicating the entire object graph, it captures primitive arrays of all flows, exploration flags, and the active node index. The `captureState` method iterates through all adjacency lists to record these values, while the `applyState` method can later re-inject these values into the graph. This mechanism is fundamental to the `HistoryManager`, ensuring that the user can navigate backward through the timeline to see precisely how the residual graph looked at any previous step.

Chapter 3

Replay Architecture and Execution Control

3.1 Snapshot-Based State Management

The ability to navigate through the history of an algorithm is implemented using a snapshot-based approach rather than a traditional command-undo pattern. Every time the algorithm reaches a significant state or performs a minor update, a new `StepSaveStruct` is created as an immutable record. This record contains a complete `GraphState` object alongside current statistics like total flow, total cost, and the specific textual explanation for that step. By saving the entire state, the application ensures that returning to a previous step is a simple matter of re-applying the saved values to the active graph, avoiding the mathematical complexity of reversing algorithmic operations.

3.2 History Navigation and Memory Efficiency

The `HistoryManager` class acts as the central repository for these snapshots, storing them in a sequential `ArrayList`. It maintains a `currentIndex` to track the user's position within the execution timeline. Navigation is handled through simple index increments or decrements; when a user clicks the "Step Back" button, the manager retrieves the previous snapshot, and the `AlgorithmController` updates the UI to match. While storing many snapshots increases memory usage, the system optimizes this by only capturing primitive data arrays from the graph rather than duplicating the entire object structure.

3.3 Threading and the Execution Engine

A significant challenge in graphical algorithm visualization is the prevention of UI freezing during long-running calculations. To resolve this, all algorithms are executed on a dedicated background thread, separate from the Swing Event Dispatch Thread (EDT). The `ExecEngine` class manages the lifecycle of this background thread using a monitor-based synchronization pattern. When an algorithm calls for a visual update, it invokes `waitIfNeeded()`, which forces the background thread to pause and enter a `wait()` state if the "paused" flag is set. This thread only resumes when it receives a `notifyAll()` signal, triggered by user actions such as clicking "Play" or "Next Step".

3.4 The Controller

The `AlgorithmController` serves as the essential bridge between the background algorithm logic and the foreground GUI components. It implements the `StepListener` interface, allowing it to receive real-time updates from any active solver. When a new step is reported, the controller manages three concurrent tasks: it updates the `HistoryManager` with a new snapshot, it dispatches a request to the `ExecEngine` to handle the necessary pause, and it uses `SwingUtilities.invokeLater` to ensure all GUI updates happen safely on the EDT. User control over speed is managed via `Thread.sleep()` in the engine or a `javax.swing.Timer` for history playback.

Chapter 4

Algorithm Implementation and Logic

4.1 Maximum Flow and the Edmonds-Karp Variation

The primary objective of the flow module is to move the largest possible volume from the source to the sink. This is achieved using the Ford-Fulkerson method, specifically the Edmonds-Karp variation which utilizes Breadth-First Search (BFS) to identify the shortest augmenting path in the residual graph. During each BFS iteration, the algorithm tracks parent nodes to reconstruct the path and maintains a record of the bottleneck capacity, which is the minimum residual capacity found along that path. Once a path is identified, the flow is updated across all involved edges, and the corresponding residual back-edges are adjusted. This process repeats until the sink is no longer reachable from the source.

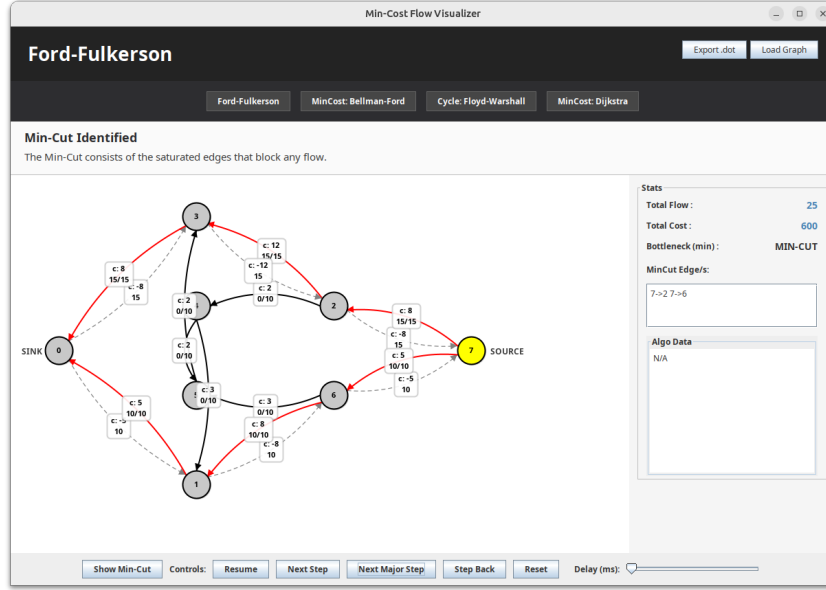


Figure 4.1: Example of FordFulkerson execution.

4.2 Minimum Cut Reachability Analysis

After the maximum flow is calculated, the system identifies the minimum cut, representing the saturated edges that limit the network's capacity. The `MinCutCalc` utility performs a final BFS starting from the source node to determine all reachable nodes in the residual graph. An edge in the original graph is identified as part of the minimum cut if it originates from a reachable node and points to an unreachable node. These edges are then flagged using the `isMinCut` property.

4.3 Cycle-Canceling for Cost Optimization

To optimize the economic cost of the flow, the project implements cycle-canceling algorithms that search for negative cost cycles in the residual graph. The `BellmanFord` implementation executes relaxation passes across all edges; if a distance improvement is detected during the final pass, it indicates a negative cycle. The algorithm traces this cycle, identifies its bottleneck, and pushes flow to reduce the total cost. Alternatively, the `FloydWarshall` solver uses an all-pairs shortest path matrix to detect these cycles by inspecting diagonal elements for negative values.

4.4 Successive Shortest Path and Node Potentials

For more efficient cost optimization, the project utilizes the Successive Shortest Path algorithm using Dijkstra's method. Because Dijkstra's algorithm cannot natively handle negative edge costs, the system implements a "node potential" system. These potentials are used to calculate reduced costs: $c_{reduced}(u, v) = c(u, v) + p(u) - p(v)$, where p represents the potential of a node. This transformation ensures that all reduced costs are non-negative, permitting the use of a priority queue. Potentials are updated after each iteration based on shortest path distances to maintain non-negativity for the next step.

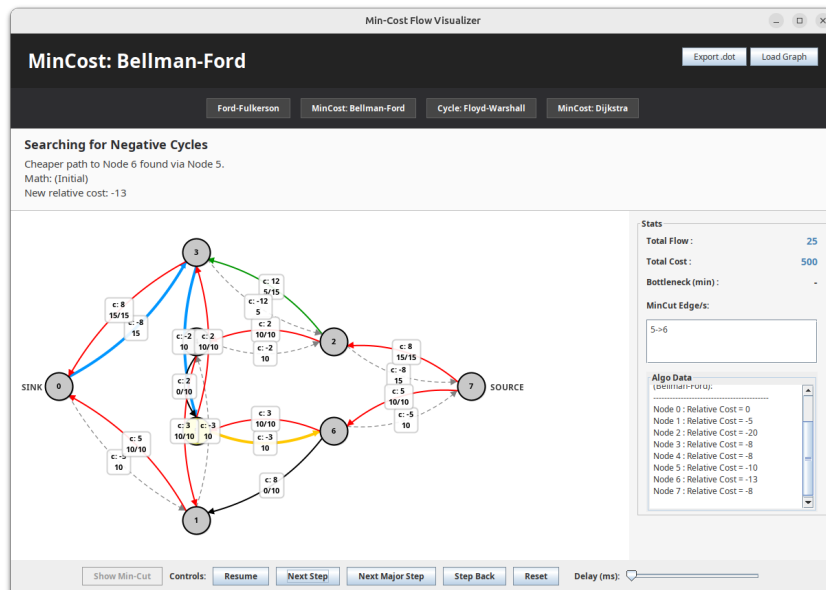


Figure 4.2: Example of BellmanFord execution.

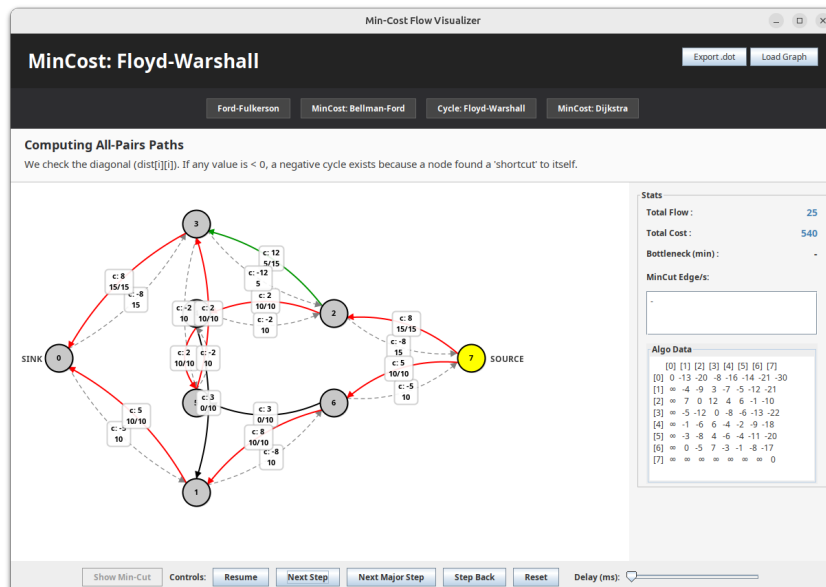


Figure 4.3: Example of FloydWarstall execution.

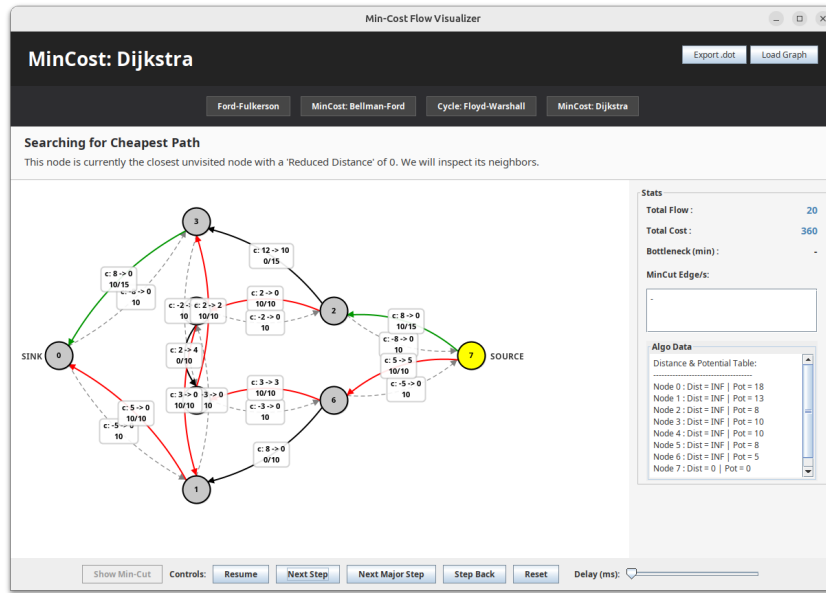


Figure 4.4: Example of Dijkstra execution.

Chapter 5

Graphical User Interface (GUI)

5.1 Modular Layout and Panel Organization

The interface is designed as a modular system using the `BorderLayout` manager to organize different functional areas within the `AppWindow`. The `HeadPanel` at the top manages file input and output, including graph loading and DOT file export. Below the header, the `MenuPanel` allows the user to select algorithms. The `InfoPanel` on the right provides continuous updates of numerical data, such as total flow, total cost, and bottleneck details. Execution is managed through the `ControlPanel` at the bottom, which contains the playback buttons and speed control.

5.2 The Graph Rendering Engine

The `GraphPainter` panel serves as the primary visualization canvas, overriding standard painting methods to render the network dynamically. To prevent visual clutter when bidirectional edges exist, the renderer utilizes `QuadCurve2D` to draw curved lines. The curvature is calculated based on node indices to ensure residual edges are always visually

distinct. Colors represent edge status: red for saturated edges, green for flow, and blue for augmenting paths. Node colors shift to yellow for active nodes and light blue for visited nodes.

5.3 Integrated Pedagogical Feedback

The `ExplanationPanel` provides a textual narrative of the algorithm's internal logic. This panel receives updates from the `AlgorithmController` whenever a step occurs, translating operations like "edge relaxation" or "bottleneck calculation" into human-readable sentences. To ensure the UI remains responsive, all updates are dispatched using `SwingUtilities.invokeLater`. This ensures synchronization between the graph visuals, the statistics in the `InfoPanel`, and the descriptions in the `ExplanationPanel`.